

Application Platform Delivery Foundations

Version 1, 2026-08-05

Introduction

Welcome to this quick course on the ***Application Platform Delivery Foundations***. This course is the **first** in a series of **six** courses about **Application Platform Delivery**:

- ***Application Platform Delivery Foundations*** (This course)
- *Cloud-Native Development with Dev Spaces*
- *Software Factory and CI/CD*
- *Application Modernization*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

Enable learners to understand the principles of Developer Experience, Platform Engineering, and GitOps-based software delivery while preparing the environment and skills required for subsequent Software Factory implementation labs.

Duration: 2 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Explain the challenges that Platform Engineering addresses and its role in improving developer experience.
- Describe the core principles of GitOps and how it enables consistent, automated application delivery.

- Identify the components of the lab environment and verify that it is ready for the exercises.
- Understand common Git branching strategies and determine when to use each approach.
- Validate essential Git operations using fundamental Git commands.
- Explain the GitHub Flow workflow for collaborative software development.
- Apply Git and GitHub best practices through hands-on lab exercises involving branching, committing, merging, and pull requests.
- Summarize the key concepts of Developer Experience, Platform Engineering, Git, and GitOps workflows.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

The Evolution of Platform Delivery

How Software Delivery Has Evolved

Modern software delivery has continuously evolved to address new challenges in application development, operations, and scalability.

This lesson explores how organizations progressed from **DevOps** to **Site Reliability Engineering (SRE)** and, ultimately, to **Platform Engineering**—the foundation of modern Application Platforms.



As you progress through this lesson, think about which stage best reflects your current organization.

It Started with DevOps

- **DevOps** transformed the way software teams collaborated by breaking down the traditional barriers between development and operations.
- Instead of working in isolation, developers and operations teams shared responsibility for delivering and operating applications.
- As organizations adopted DevOps practices, developers gradually assumed additional operational responsibilities beyond writing application code.

▼ What was expected from developers? (Click to expand)

Developers were expected to:

- Write application code
- Build and manage infrastructure
- Configure CI/CD pipelines
- Deploy applications
- Monitor production systems
- Respond to incidents
- Understand Kubernetes and cloud platforms



While DevOps increased delivery speed, it also introduced a new challenge—**developer cognitive overload**. Developers spent more time managing infrastructure and less time building business features.

Next Came Site Reliability Engineering (SRE)

- As applications became more complex, organizations introduced **Site Reliability Engineering (SRE)** to improve reliability through automation, monitoring, and operational best practices.
- SRE teams helped automate operational tasks and improve platform reliability. However, as the

number of applications and development teams grew, centralized SRE teams often became a bottleneck because every team depended on their expertise.

▼ *Common SRE responsibilities (Click to expand)*

- Automating operational tasks
- Improving reliability
- Monitoring production systems
- Incident response
- Performance optimization
- Capacity planning

Then Came Platform Engineering

- **Platform Engineering** emerged to reduce developer cognitive load while enabling organizations to standardize software delivery.
- Instead of expecting every developer to become an infrastructure expert—or every SRE team to support every application—platform teams build reusable capabilities that developers can consume through self-service experiences.
- These capabilities include:
 - Self-service developer experiences
 - Standardized workflows
 - Golden Paths
 - Internal Developer Platforms (IDPs)
 - Built-in security and governance
 - Automated software delivery

The Market Opportunity

80%

of software engineering organizations will establish platform teams by 2026 (Gartner).



Drive Velocity: Reduce cognitive load.



Scale Best Practices: Standardize security & compliance.



Developer Experience: The new competitive advantage.

▼ *Characteristics of Platform Engineering (Click to expand)*

Platform Engineering treats the platform as a product.

Rather than building applications directly, Platform Engineers build reusable capabilities that application teams can consume on demand.

Their goal is to make the secure, compliant, and recommended path the easiest path for developers.

Key Takeaway

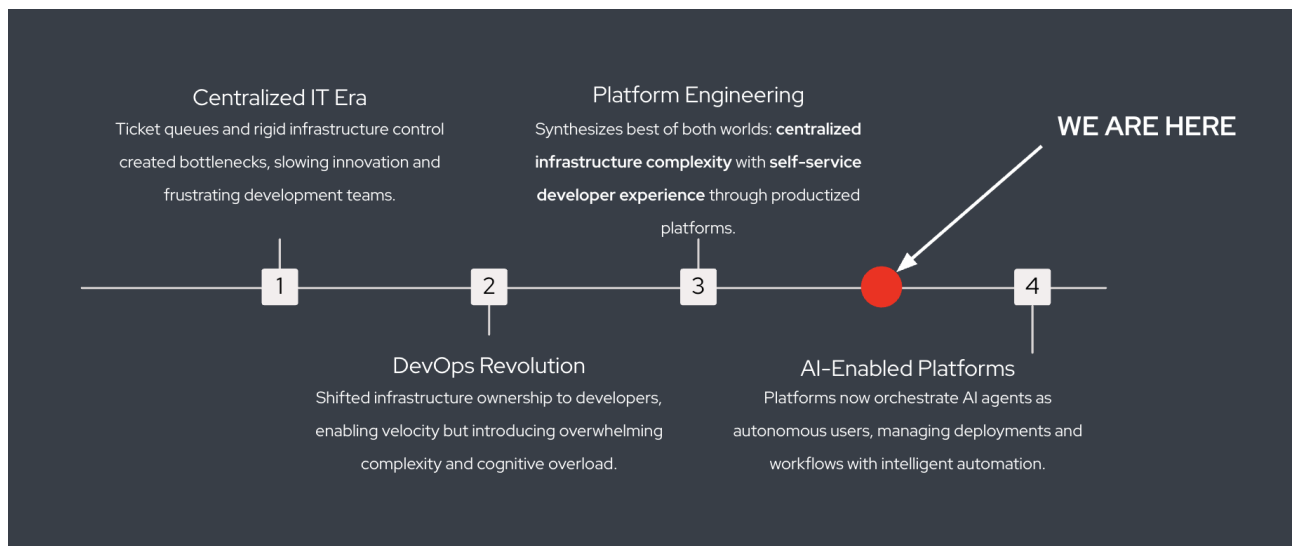
Platform Engineering doesn't replace DevOps or SRE.

Instead, it builds upon their strengths by creating reusable platforms that enable developers to deliver software faster while allowing operations teams to maintain security, governance, and reliability.



The Evolution of Platform Delivery

- Software delivery continues to evolve.



▼ *Traditional IT (Click to expand)*

Infrastructure provisioning was centralized.

Developers relied heavily on operations teams to provision servers, configure environments, and deploy applications.

▼ *DevOps (Click to expand)*

Developers gained greater ownership of the application lifecycle.

This increased agility but also increased operational complexity and cognitive load.

▼ *Platform Engineering (Click to expand)*

Platform teams provide reusable services, Golden Paths, and Internal Developer Platforms (IDPs).

Developers consume these services through self-service workflows while platform teams maintain governance and standardization.

▼ *AI-Assisted Development (Click to expand)*

Modern platforms are beginning to support AI assistants and autonomous agents alongside human developers.

These platforms provide APIs, guardrails, and automation that enable both developers and AI to build, test, deploy, and operate applications safely.



Throughout this learning path, we'll focus primarily on **Platform Engineering** because it forms the foundation of modern Application Platforms such as Red Hat OpenShift.

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Which stage best matches each statement?

1. Developers own both application code and infrastructure.
2. Platform teams provide Golden Paths and self-service capabilities.
3. Operations teams provision infrastructure for every application.
4. AI assistants begin interacting directly with the platform.

Answers

1. DevOps
2. Platform Engineering
3. Traditional IT
4. AI-Assisted Development

Diving into Application Platform

In this lesson, you'll learn why Kubernetes alone isn't enough and how an **Application Platform** provides the capabilities organizations need to build, deliver, secure, and operate modern applications.



As you progress through this lesson, think about the tools your organization currently uses. Which capabilities are already integrated, and which ones require separate products?

Why Isn't Kubernetes Alone Enough?

- Many organizations begin their cloud-native journey by adopting Kubernetes or a managed Kubernetes service.
- Kubernetes excels at orchestrating containerized workloads, but it doesn't provide everything required to support the complete software development lifecycle.
- As organizations scale, development teams also require capabilities such as:
 - Application runtimes
 - Build services
 - CI/CD pipelines
 - GitOps
 - Container registries
 - Observability
 - Security and compliance
 - Identity and access management
 - Developer tools
- Without an integrated platform, organizations must select, integrate, secure, upgrade, and maintain each of these capabilities independently. This increases operational complexity and creates inconsistent developer experiences.



Kubernetes answers the question: "How do I run containers?"

An Application Platform answers the broader question: "How do I efficiently build, deliver, secure, and operate applications?"

What is an Application Platform?

- An **Application Platform** brings together the capabilities required to support the complete software development lifecycle.
- Rather than providing isolated tools, it delivers an integrated developer experience that enables organizations to standardize software delivery while allowing developers to focus on building

applications.

Application Build	 Runtimes & languages	 Developer Tools	 Serverless & functions	 Cloud Services APIs	 Migration tools
Application Delivery	 CI/CD & GitOps	 Container registry	 Observability	 Logging	 Virtual machines
Application Security	 Service mesh	 Vulnerability scanning	 Security policies	 Cert & secret tooling	 Auth and SSO services



Click each capability area below to explore the services typically provided by an Application Platform.

▼ *Application Build (Click to expand)*

Application Build provides everything developers need to design, develop, modernize, and package applications.

Typical capabilities include:

- Application runtimes
- Developer tools
- Serverless platforms
- Cloud service integrations
- Migration tooling

These capabilities provide standardized development environments that help teams build applications faster while following consistent engineering practices.

▼ *Application Delivery (Click to expand)*

Application Delivery automates how applications are built, tested, deployed, monitored, and managed throughout their lifecycle.

Typical capabilities include:

- CI/CD pipelines
- GitOps
- Container registries
- Observability
- Logging
- Virtual machine support

Together, these capabilities improve deployment consistency while reducing operational effort.

▼ *Application Security (Click to expand)*

Application Security embeds security throughout the software delivery lifecycle instead of treating it as a separate activity.

Typical capabilities include:

- Service Mesh
- Vulnerability scanning
- Security policy enforcement
- Certificate and secret management
- Single Sign-On (SSO)

By integrating security into the platform, organizations can deliver applications that are secure, compliant, and enterprise-ready.



Key Takeaway

An **Application Platform** combines application development, delivery, and security capabilities into a unified developer experience, allowing teams to focus on delivering software rather than integrating and maintaining individual tools.

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Which statement best describes an Application Platform?

1. A platform that only manages containers.
2. A collection of unrelated development tools.
3. A unified platform that provides integrated capabilities for building, delivering, securing, and operating applications.
4. A virtual machine management platform.

Answer

C — An Application Platform provides an integrated set of capabilities that support the complete software development lifecycle.

Container Management vs Application Platform

This lesson explores the difference between a **Container Management Platform** and an **Application Platform**.

Understanding the Difference

- Kubernetes revolutionized application deployment by providing powerful capabilities for container orchestration, scheduling, scaling, and workload management.
- However, enterprise software delivery involves much more than running containers.
- Development teams also require capabilities such as application builds, CI/CD, GitOps, observability, security, developer self-service, and standardized workflows.
- A **Container Management Platform** provides the infrastructure required to run containerized workloads.
- An **Application Platform** builds on that foundation by integrating the capabilities required to build, deliver, secure, and operate applications throughout their lifecycle.



A **Container Management Platform** focuses on **running containers**.

An **Application Platform** focuses on **delivering applications**.

Comparing the Two Approaches

The following table highlights the key differences between the two approaches.

Feature	Container Management Platform	Application Platform
Primary Purpose	Deploy, orchestrate, and manage containerized workloads.	Build, deliver, secure, and operate applications throughout their lifecycle.
Primary Focus	Infrastructure and container orchestration.	Application development and developer productivity.
Starting Point	Deploy existing container images.	Develop, build, and deploy applications directly from source code.
Developer Experience	Developers manage infrastructure resources and deployment configurations.	Developers consume self-service platform capabilities through standardized workflows.
Operational Responsibility	Platform teams assemble and maintain multiple supporting tools.	Integrated platform capabilities reduce operational complexity.
Platform Engineering	Focuses on integrating independent products.	Focuses on creating reusable platforms, Golden Paths, and developer self-service.

Feature	Container Management Platform	Application Platform
Application Networking	Advanced networking and security require significant manual configuration.	Built-in routing, Service Mesh, and security simplify modern application delivery.
Enterprise Readiness	Provides the foundation for running containers.	Provides an integrated enterprise application platform.

Why Organizations Choose an Application Platform

As organizations grow, maintaining independent tools for builds, deployments, security, monitoring, networking, and developer onboarding becomes increasingly complex.

An Application Platform simplifies this by providing:

- Integrated developer tools
- Standardized application runtimes
- Automated CI/CD and GitOps
- Built-in security and governance
- Observability and monitoring
- Self-service developer experiences
- Reusable Golden Paths

Instead of assembling and maintaining dozens of independent tools, platform teams provide developers with a consistent, enterprise-ready platform.



The goal of an Application Platform isn't to replace Kubernetes—it extends Kubernetes with the capabilities required for modern software delivery.

Key Takeaway

A **Container Management Platform** answers the question:



- "How do I run containers?"

An **Application Platform** answers the broader question:

- "How do I efficiently build, deliver, secure, and operate applications?"

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Which platform best fits each requirement?

Requirement	Platform
Standardized developer workflows	?
Container orchestration	?
Built-in CI/CD and GitOps	?
Self-service developer experience	?
Running containerized workloads	?

Answers

- Standardized developer workflows → Application Platform
- Container orchestration → Container Management Platform
- Built-in CI/CD and GitOps → Application Platform
- Self-service developer experience → Application Platform
- Running containerized workloads → Container Management Platform

Delivering Application Platform

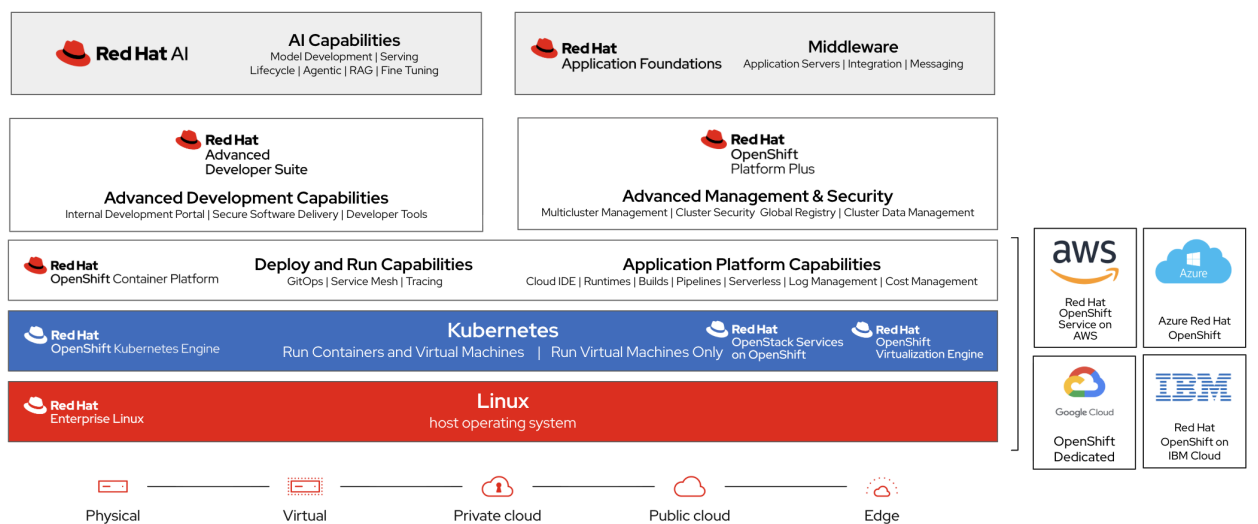
In this lesson, you'll explore how **Red Hat OpenShift** extends Kubernetes to deliver a complete **Application Platform**. You'll learn how OpenShift integrates application development, delivery, security, and platform engineering capabilities into a unified platform that supports the entire application lifecycle.



As you progress through this lesson, identify the capabilities that transform Kubernetes into a complete Application Platform. Consider how integrating these capabilities into a single platform can simplify software delivery and improve the developer experience.

How Red Hat OpenShift Delivers an Application Platform

- **Red Hat OpenShift** builds on Kubernetes by integrating the capabilities required to support the complete application lifecycle.
- Rather than requiring organizations to assemble and maintain independent products, OpenShift provides a unified platform that includes developer tools, application runtimes, build services, CI/CD, GitOps, security, AI capabilities, and hybrid cloud support.
- The following diagram illustrates how these capabilities are layered on top of Kubernetes to create a complete Application Platform.



▼ OpenShift Capability Map (Click to expand)

At the foundation, **Red Hat Enterprise Linux** provides the operating system, while **Kubernetes** delivers container orchestration and workload management.

OpenShift extends this foundation by integrating capabilities for:

- Application development
- Application builds

- CI/CD and GitOps
- Serverless
- Service Mesh
- Developer tools
- Internal Developer Platforms
- Security
- AI services
- Multicloud and hybrid cloud management

Together, these capabilities provide a consistent platform for building, deploying, securing, and operating applications across on-premises, public cloud, private cloud, virtualized environments, and edge locations.

Reflection

▼ *Think about your own organization (Click to expand)*

Reflect on your current software delivery platform.

- Which application platform capabilities are available out of the box?
- Which capabilities require separate products or manual integration?
- How much effort is spent maintaining those integrations?
- How could an integrated Application Platform simplify software delivery for your teams?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Your organization wants to provide developers with:

- Standardized application runtimes
- Automated CI/CD and GitOps
- Integrated security
- Built-in observability
- Self-service developer workflows

Which approach would best support these requirements?

1. Kubernetes with separately integrated tools
2. Red Hat OpenShift

Answer

B — Red Hat OpenShift provides these capabilities as an integrated Application Platform built on Kubernetes, reducing operational complexity while improving developer productivity.

Application Platform Litmus Test

In this lesson, you'll learn a simple way to evaluate whether a solution qualifies as a modern **Application Platform**.



As you progress through this lesson, think about your own organization. Does your current platform provide these capabilities as an integrated experience or through separate products?

The Four Essential Capabilities

Every modern **Application Platform** should provide the following foundational capabilities.

Capability	Why It Matters
Application Runtimes	Provides standardized environments for developing and running applications.
Application Builds	Builds and packages applications into deployable artifacts.
Application Delivery (CI/CD)	Automates building, testing, deploying, and promoting applications across environments.
Observability	Provides integrated monitoring, logging, and tracing to understand application health and troubleshoot issues.

These capabilities form the technical foundation of an Application Platform. While organizations can assemble them using independent tools, an Application Platform delivers them as a unified developer experience.

Beyond Individual Tools

Having the right capabilities is only part of the story.

The real value of an **Application Platform** comes from how those capabilities are integrated and delivered to developers.

Instead of every development team selecting, configuring, and maintaining its own toolchain, the platform provides:

- Standardized development environments
- Automated build and deployment pipelines
- Integrated security and policy enforcement
- Built-in observability
- Self-service templates and Golden Paths

By providing these capabilities through a consistent platform, organizations improve developer productivity, reduce operational complexity, and establish standardized engineering practices

across teams.

Reflection

▼ *Think About Your Organization (Click to expand)*

Consider your current software delivery platform.

- Which of the four foundational capabilities are already integrated?
- Which capabilities require separate products or manual integration?
- Does your platform provide standardized developer workflows?
- If a new developer joined your team today, how quickly could they build and deploy an application?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

A platform provides the following capabilities:

- Kubernetes
- Application runtimes
- CI/CD pipelines
- Observability

However, each development team is responsible for integrating its own security tools, deployment workflows, and developer onboarding process.

Would you consider this a complete **Application Platform**?

1. Yes
2. No

Answer

B — Although several foundational capabilities are available, a complete Application Platform also provides an integrated developer experience with standardized workflows, built-in automation, and organizational guardrails.

Meet the Teams

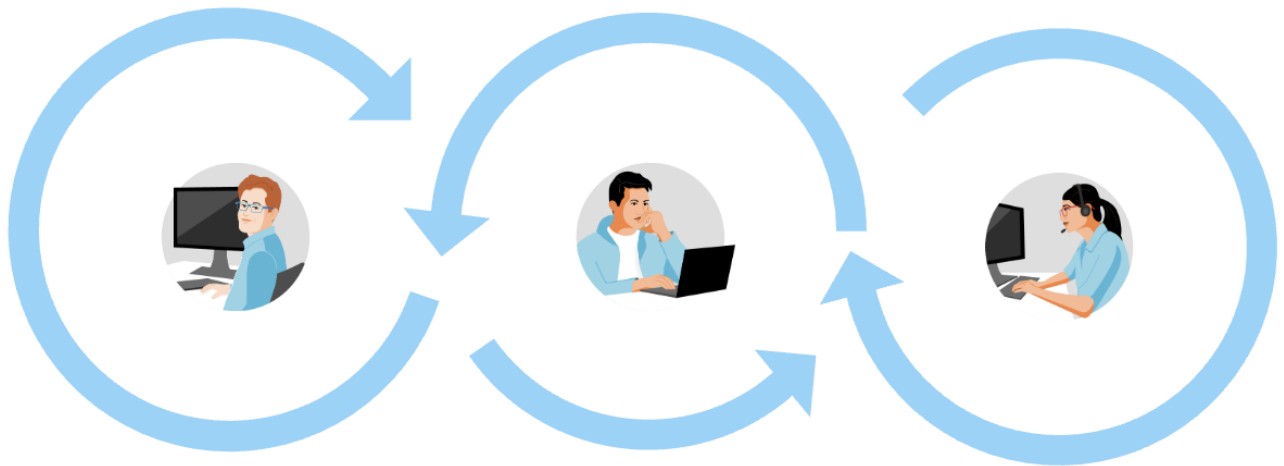
In this lesson, you'll meet the three primary personas that interact with an Application Platform and understand how each team contributes to modern software delivery.



As you progress through this lesson, think about your own organization. Which team do you work with most closely, and what challenges do they face when building, deploying, or operating applications?

Why Different Teams Matter

- Modern software delivery is a collaborative effort.
- Rather than being owned by a single team, building and operating applications requires close collaboration between **Operations Engineers**, **Platform Engineers**, and **Application Developers**. Although they share the same business goals, each team has different responsibilities, priorities, and measures of success.



Throughout this learning path, you'll revisit these personas to understand how different Application Platform capabilities help solve their day-to-day challenges.

Meet Alex — Operations Engineer



- Alex is responsible for the reliability, security, and day-to-day operation of the platform.

- His work ensures that developers have a stable and secure environment for running applications.
- Success is measured by:
 - Platform availability
 - Reliability
 - Security
 - Compliance
 - Operational stability

▼ *Common questions Alex asks (Click to expand)*

- How do I keep the platform secure?
- How do I perform upgrades with minimal downtime?
- How do I enforce organizational policies?
- How can I monitor cluster health and reliability?



Alex wants developers to move quickly—but never at the expense of security or reliability.

Meet Penny — Platform Engineer



- Penny builds and maintains the Internal Developer Platform used by application teams.
- Rather than provisioning infrastructure for every request, she creates reusable templates, Golden Paths, and self-service capabilities that simplify software delivery while enforcing organizational standards.
- Success is measured by:
 - Developer productivity
 - Platform adoption
 - Golden Paths
 - Standardization

- Developer satisfaction

▼ *Common questions Penny asks (Click to expand)*

- How can developers onboard faster?
- How do I reduce repetitive operational work?
- How can I provide secure defaults?
- How do I standardize software delivery across teams?



Penny's goal is to make the recommended path the easiest path for developers.

Meet David — Application Developer



- David develops application features that deliver business value.
- He wants to spend his time writing code rather than managing infrastructure, configuring Kubernetes resources, or maintaining deployment pipelines.
- Success is measured by:
 - Shipping features quickly
 - Fast onboarding
 - Reduced cognitive load
 - Development velocity

▼ *Common questions David asks (Click to expand)*

- How quickly can I deploy my application?
- Do I need to understand Kubernetes?
- Can I create new services without opening infrastructure tickets?
- How can I spend more time writing code and less time managing platform configuration?



David wants to focus on building applications—not managing infrastructure.

Comparing the Personas

Although each team has different responsibilities, they all rely on the same Application Platform.

Operations Engineer	Platform Engineer	Application Developer
Reliability	Developer Experience	Feature Delivery
Security	Standardization	Fast Onboarding
Compliance	Self-Service	Productivity
Platform Availability	Golden Paths	Rapid Delivery

Why an Application Platform Matters

- Without an integrated Application Platform, each team spends significant effort selecting, integrating, and maintaining independent tools for building, deploying, securing, and operating applications.
- An Application Platform reduces this complexity by providing a consistent developer experience that benefits every persona:
 - Operations Engineers manage secure and reliable platforms.
 - Platform Engineers provide reusable services and organizational guardrails.
 - Application Developers consume self-service capabilities and focus on delivering business value.

Reflection

▼ Think About Your Organization (Click to expand)

Consider how software is delivered in your organization.

- Which persona best matches your current role?
- Which team owns the developer platform?
- Where do developers experience the most friction?
- Which persona would benefit the most from an Application Platform?

Check Your Understanding

▼ Quiz Time! (Click to expand)

Match each responsibility with the persona most likely responsible for it.

1. Creating reusable Golden Paths
2. Maintaining platform availability
3. Delivering new application features
4. Improving developer onboarding

5. Enforcing organizational security standards

Answers

1. Platform Engineer
2. Operations Engineer
3. Application Developer
4. Platform Engineer
5. Operations Engineer

Summary

What's next?

References

- [Optimize application development overview](#)
- [Red Hat Developer Experience Assessment: empower Dev teams](#)
- [Developer Experience Assessment at Red Hat Sales Hub](#)
- [Developer Experience Assessment Miro Board Template](#)
- [Red Hat Advanced Developer Suite - Product Page](#)
- [Red Hat Advanced Developer Suite - RHCC](#)